

Exp No : 10 Malware Disassembly and Code Analysis Using Ghidra

DATE:

1. AIM

To set up a secure analysis environment, utilize the Ghidra reverse engineering framework to disassemble and decompile binary code, and interpret low-level assembly functions to identify common malware behaviors, persistence mechanisms, and indicators of compromise.

2. SOFTWARE / TOOLS REQUIRED

- **Operating System:** Isolated Virtual Machine (Windows/Linux OS)
- **Forensics Tool:** Ghidra (Open-source software reverse engineering suite developed by the NSA)
- **Runtime Environment:** Java Development Kit (JDK) required for running Ghidra
- **Sample Files:** Benign sample binaries (benign_sample1.bin, benign_sample2.hex) or controlled test binaries exhibiting typical executable structures.

3. DESCRIPTION

Malware analysis is the critical process of dissecting compiled software to understand its capabilities, origins, and operational intent without executing it live (static analysis). Ghidra, an open-source Software Reverse Engineering framework developed by the National Security Agency (NSA), is utilized extensively by forensic investigators for this purpose. When original source code is unavailable or intentionally obfuscated by adversaries, analysts rely on Ghidra to bridge the gap between raw, machine-executable binaries and human-readable logic.

The framework's core strength lies in its CodeBrowser, which parses raw binary instructions into low-level assembly language, while its advanced Decompiler algorithmically reconstructs higher-level, C-like pseudo-code. This dual approach significantly reduces the cognitive load required to decipher complex malware algorithms. To further aid in understanding the execution logic, Ghidra generates interactive Control Flow Graphs (CFGs) that visually map out function calls, decision branches, and loops, allowing analysts to trace execution paths and identify evasion techniques.

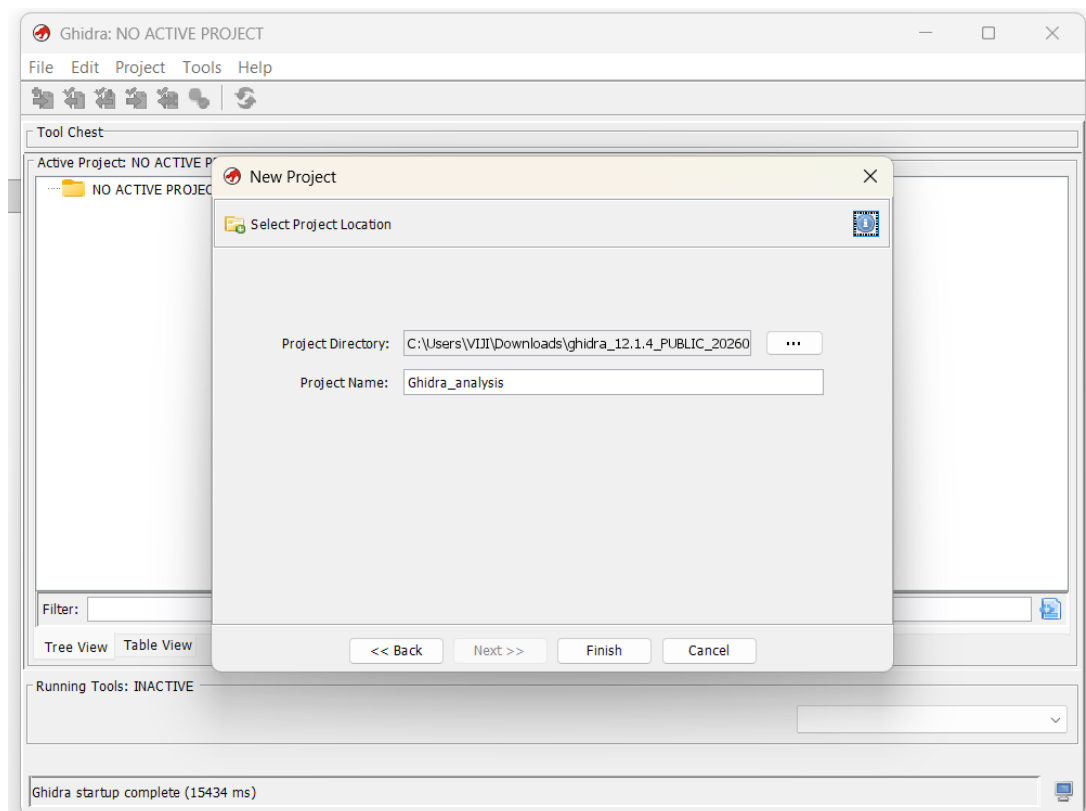
4. PROCEDURE

1. Open Ghidra

- Open the extracted Ghidra folder.
- Run "ghidraRun.bat".
- Wait for the Ghidra Project window to open.

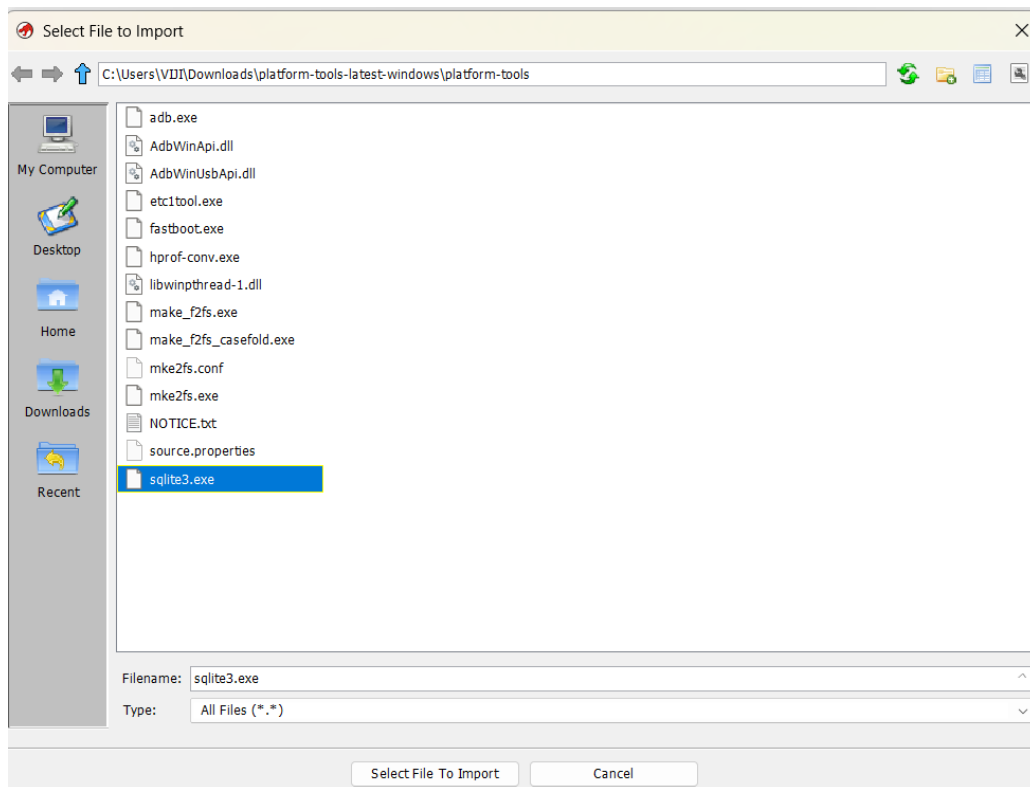
2. Create a Project

- Select File → New Project.
- Choose Non-Shared Project.
- Enter the project name and click Finish.



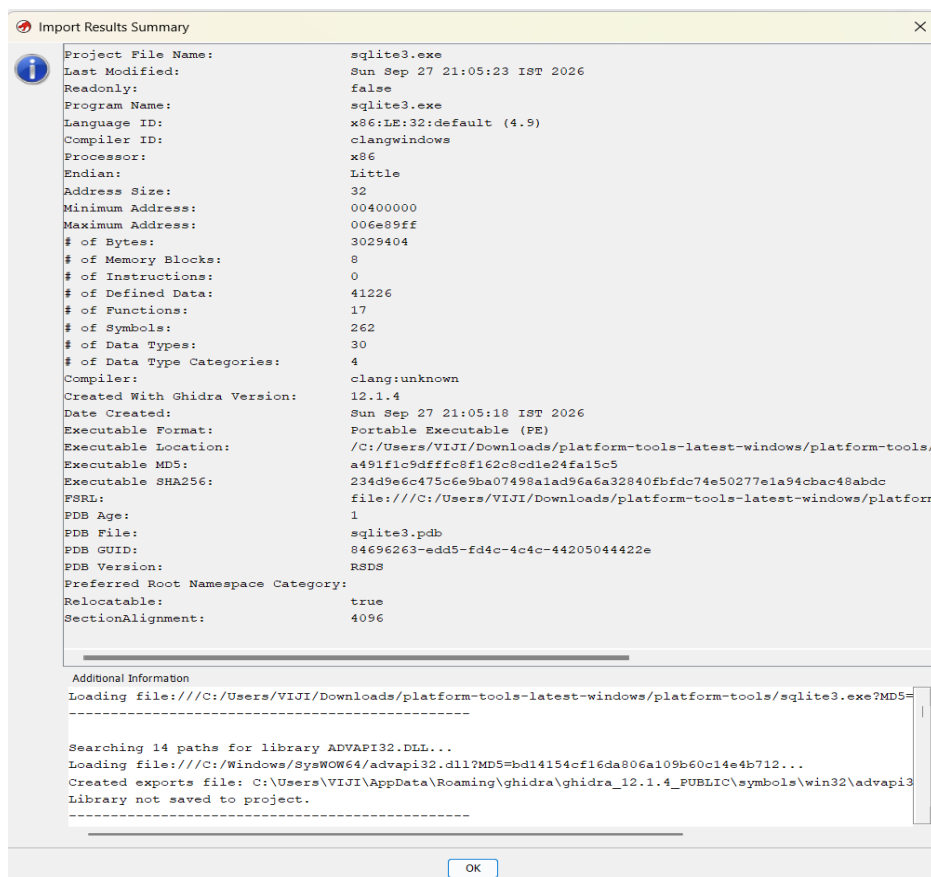
3. Import Binary

- Select File → Import File.
- Choose the given benign binary.
- Click OK to import the file.



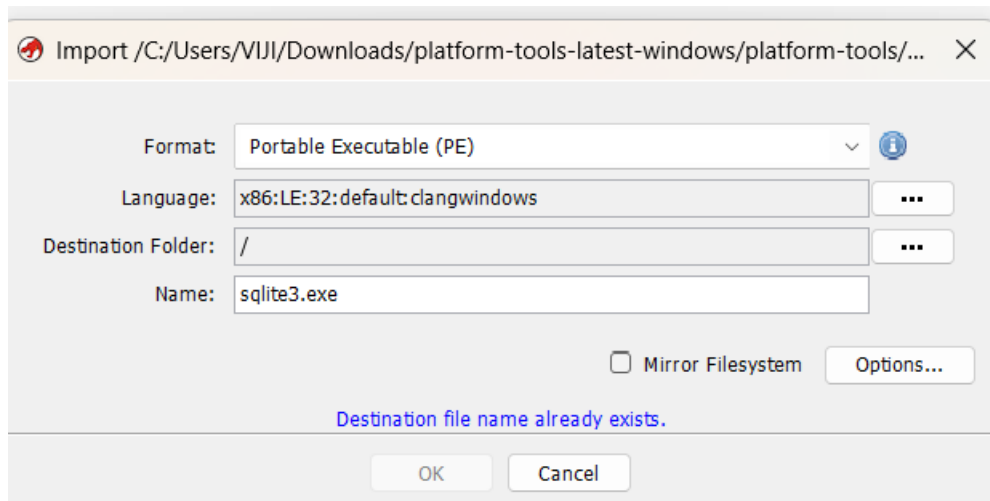
4. Run Auto Analysis

- Open the imported binary in CodeBrowser.
- Select Yes when asked to analyze the program.
- Click Analyze using the default options.



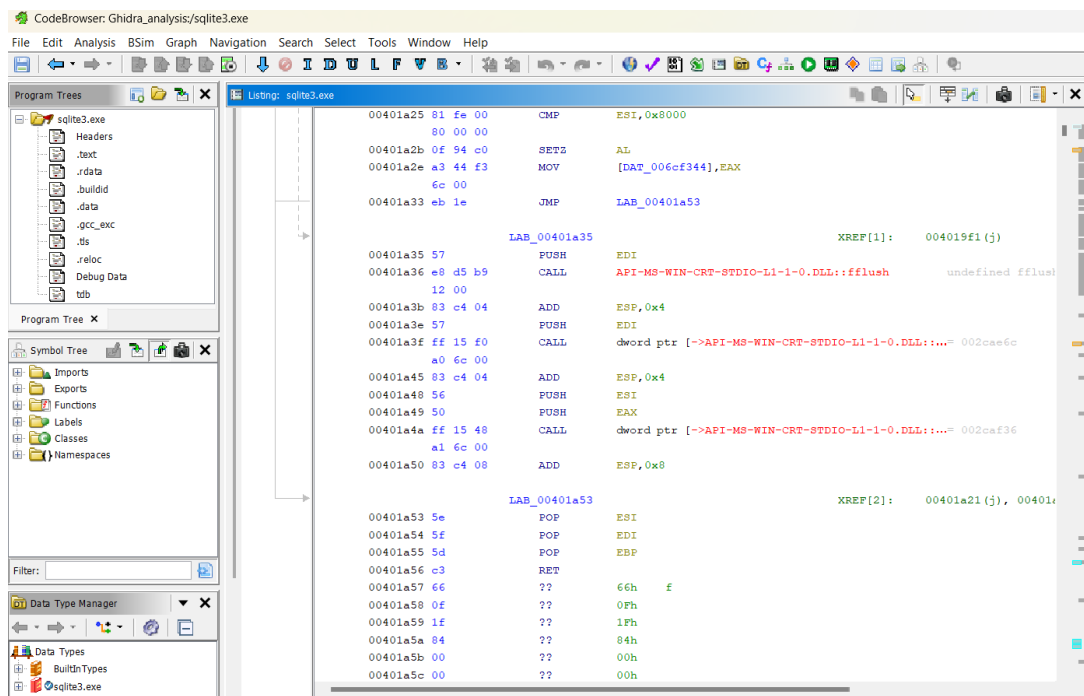
5. Identify Functions

- Open Symbol Tree → Functions.
- Identify functions such as "main", "entry", or "FUN_...".
- Select the required function for analysis.



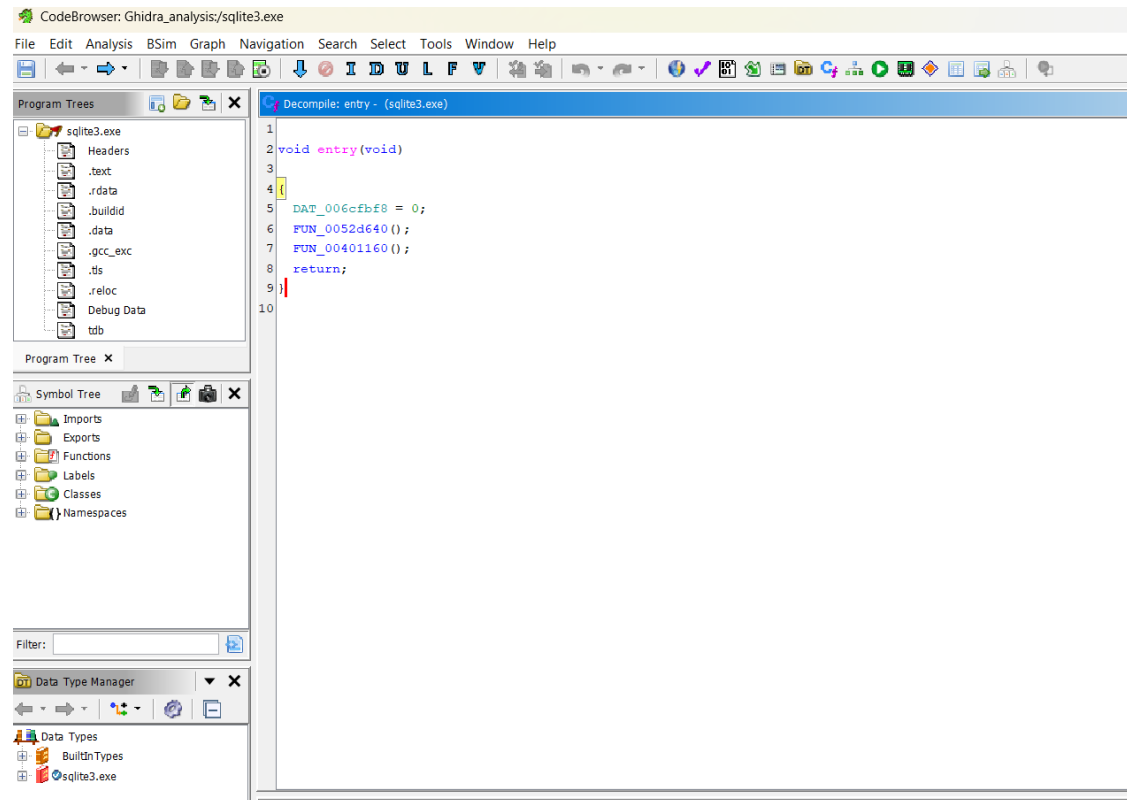
6. Analyze Functions

- Examine the assembly instructions of the selected function.
- Observe function calls and cross-references.
- Identify the purpose of the function.



7. Use Decompiler

- Open the Decompiler window.
- Select the required function.
- Examine the generated C-like code.



8. Analyze Control Flow

- Open the Function Graph.
- Observe blocks, branches, and connections.
- Understand the execution flow of the function.

9. Document Findings

- Record important functions, strings, and imports.
- Note the observed program behavior.
- Save screenshots and document the analysis results.

RUBRICS

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

5. RESULT

Ghidra was successfully utilized to disassemble, decompile, and analyze the target binary code. Through automated analysis, string inspection, API import tracking, and decompiler evaluation, the binary's underlying structure and behavioral logic were effectively interpreted.